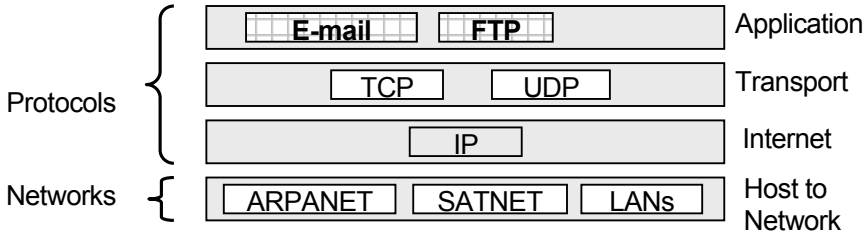
Contents of lecture 7:

1. **File Transfer Protocol (FTP)**
   - Details of how FTP operates
2. **Electronic Mail**
   - E-mail formatting + extensions
   - Sending E-mails
   - Collecting E-mails

The figure opposite places the content of this lecture in relation to the wider TCP/IP model:



File Transfer Protocol (FTP)

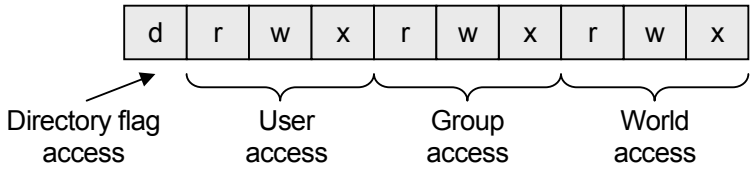
Precursor: Access control

FTP is a protocol used to transfer files to and from a particular network host. Whilst this is a very useful ability, it is evident the openness of the service must be highly constrained in order to prevent misuse.

At the broadest level, FTP access to a particular machine can be disabled; however, this excludes advantageous use of FTP as well as unwanted usage.

Because of this machines that support FTP provide file and directory access control. Hence, a user can retain full access to their files, permit group members to read (but not write/delete) their files, and deny access to all other people.

Access control is normally accomplished by associating a number of access flags with each file and directory (e.g. a read-only flag, etc.). How this is done is OS specific, however, the most commonly encountered using FTP is UNIX based and is shown below:



Three groups of access flags are provided: for the user, their workgroup and then general access. Three flags are included within each group: one for read access, another for write access, and a third for execute privileges.

File Transfer Protocol (FTP)

FTP (File Transfer Protocol) is defined in the RFC959 document, which can be found at the following location:

ftp://nic.merit.edu/documents/rfc/rfc0959.txt

The design objectives of FTP were as follows:

1. To promote sharing of files (computer programs or data).
2. To encourage indirect or implicit use of remote computers (i.e. to be a network based application that can be readily employed).
3. To shield a user from variations in file storage systems among hosts.
4. To transfer data reliably and efficiently.

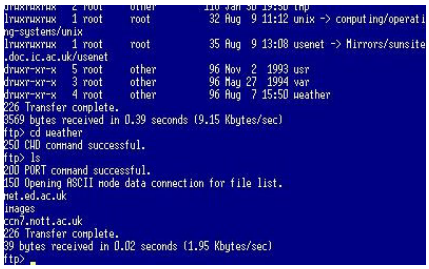
Importantly, RFC959 also notes that FTP, whilst being directly usable by the user, is designed mainly for use within programs, i.e. a program provides an easy interface through which the FTP protocol may be used.

Functionality of FTP

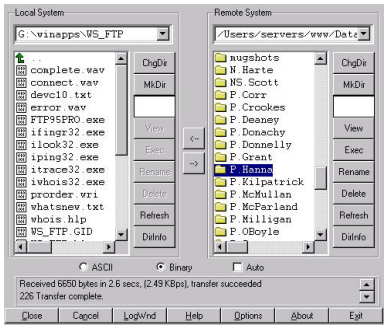
TCP is a client-server process, whereby an FTP server (a daemon listening for incoming FTP requests/commands) interacts with FTP clients.

In essence, FTP provides a means of transferring a file between two computers, however, most FTP clients also permit the user to navigate the file system, create/delete directories/files (if permitted) on both the local and connected machines, etc.

A number of more recent FTP clients also offer repeated download attempts, re-connection should the connection be lost, resumed downloads, etc. An example of a basic FTP client follow:



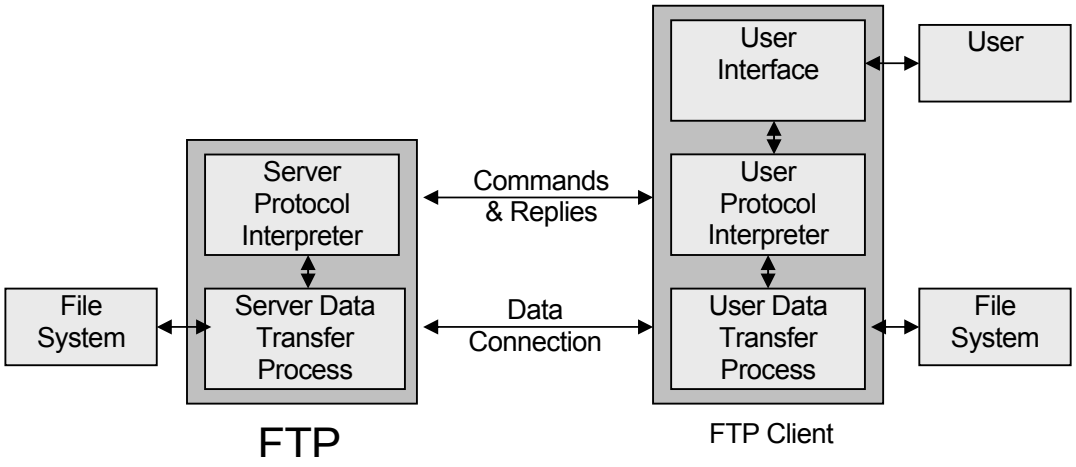
The client shown below is more modern:



FTP was first proposed in 1971, with the current standard (introduced in 1985) defined with international document RFC959.

Functional breakdown of FTP

In terms of functional units, FTP can be viewed as follows:



The FTP client consists of a *User Protocol Interpreter* (PI) that initiates a connection with the *Server Protocol Interpreter* (using port 21). The user PI sends FTP commands to the server PI, which in turn replies back to the user PI.

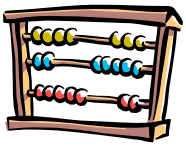
The user and server data transfer processes (operating on port 20) are used to transfer data between the client and server. Note, the connection is duplex, and is only established whenever some data needs to be sent (the connection is relinquished whenever the data transfer has completed).

The FTP client will also contain a user interface, permitting the user to specify which FTP commands are to be sent by the user PI to the server PI.

Data Representation and Storage

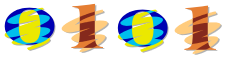
FTP provides a number of different mechanisms through which differing data formats can be integrated, i.e. one computer might use a different format for storing text files than that found on another (full details found within RFC959).

Thankfully, most of the features supported within the FTP specification have been rendered defunct given the greater degree of standardisation within the computer field since 1985 (when the RFC959 proposal was developed), i.e. many of the file formats supported within FTP are now not used.



Nowadays, only two different data formats are widely used as follows:

- ASCII/Text only: This is the default FTP format, and stipulates that all data is encoded as 8-bit ASCII.
- Binary/Image: Data is encoded as contiguous 8-bit bytes.



ASCII and binary formats differ in that if the data is classified as ASCII then certain bytes will be interpreted as end-of-line markers, etc.

Miscellaneous FTP details

- By default, FTP sessions are established on port 21, although the server may be configured to accept connections on a different port.
- It is the server's responsibility to initate the data connection, to maintain it and to close it. Furthermore, all data transfers must be accomplished with an end-of-file (EOF), which may be explicitly send, or implied if the data connection is closed.
- FTP servers normally have a timeout period, usually a few minutes. The timeout period is reset each time the client sends a command. The client is disconnected if the timeout expires.

FTP Commands

The following list of commands are those employed within the User and Server Protocol Interpreters (PIs). Typically the PI commands are more primitive than the commands that the user interface offers, i.e. FTP clients provide a simplified interface. However, the user can directly issue FTP commands if so desired.

- **USER** – *User name*: This command is normally the first one sent to the server after the control connection has been made, and is used to identify the user to the server, and hence the degree of file access they command. The username is passed as the argument.
- **PASS** – *Password*: This command must be sent immediately after the user name command and for most sites completes the user's identification. It is the responsibility of the FTP client to ensure that sensitive password information is protected (i.e. not echoed to the screen, etc.). The password is passed as the argument.

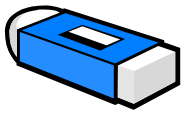
- **CWD** – *Change Working Directory*: This command permits the user to change to a different directory for file storage and retrieval, assuming sufficient access privileges. The desired working directory is passed as the argument.
- **CDUP** – *Change To Parent Directory* – This command is a special case of CWD, intended to simplify the process of transferring directory trees between operating systems having different syntaxes for naming the parent directory (assuming the user has sufficient access privileges).

- **REIN** – *Reinitialise*: This command terminates the user's connection, allowing the current file transfer, if any, to complete. All parameters are then reset to the default setting, i.e. equivalent to the state the user finds themselves in upon first connection.
- **QUIT** – *Logout*: This command terminates the user's connection, with the server closing any file transfers currently in progress.
- **PORT** – *Data Port*: This command permits the port used for data transfer to be specified. The command accepts as arguments a 32-bit internet host address, and a 16-bit port address.
- **TYPE** – *Representation Type*: The argument to this command specifies the representation of the data, usually either A – ASCII or I – Image.

- **RETR** – *Retrieve*: This command causes the server data transfer process (DTP) to transfer a copy of the specified file to the user DTP (assuming the user has sufficient access privileges).

- **STOR** – *Store*: This command causes the server DTP to accept the data transferred from the user DTP and to store the data server side (assuming the user has sufficient access privileges).

- **STOU** – *Store Unique*: This command is the same as STOR, except that the file to be stored by the server must be generated using a name that is unique to the directory within which it is to be stored. The server's reply must include the name of the created file.
- **APPE** – *Append*: This command causes the server to accept data from the client, and store it in the specified file, appending it to the end of the file if it already exists.
- **RNFR** – *Rename From*: This command specifies the name of a file that is to be remained. It is used in conjunction with RNTO
- **RNTO** – *Rename To*: This command takes the file identified by the RNFR command and assignes it the new specified filename (provided the user has sufficient access privileges).
- **ABOR** – *Abort*: This command tells the server to abort the previous FTP command and any associated data transfer.
- **DELE** – *Delete*: This command causes the specified file to be deleted on the server (assuming the user has sufficient access privileges).



- **MKD** – *Make Directory*: This command causes the directory specified as the argument to be created (assuming the user has sufficient access privileges).
- **PWD** – *Print Working Directory*: This command causes the name of the current working directory to be returned.
- **LIST** – *List*: This command causes a list of all the files and directories within the current working directory to be sent to the client.
- **HELP** – *Help*: This command results in the server sending helpful information regarding its implementation, etc.

- **NOOP** – *No Operation*: This command specifies that the server sends an OK reply – most often used to keep the FTP connection 'alive'.

FTP Replies

The FTP server sends replies back to the FTP client, providing information about the last command. Replies assume the following format:

3 digit number	Textual response

The three digit number uniquely identifies the reply returned by the FTP server (and the FTP client bases its response upon the received digits), i.e. it is intended that the three digit number contain sufficient encoded information that the user PI need not examine the appended text. The number is normally followed by some useful textual information, which is intended for the user.

From time-to-time it will be necessary for the server to send a multi-line response. The FTP protocol specifies that this is accomplished by immediately following the 3 digit code by a hyphen, and then signifying the end of the multi-line message by sending the same three digit code again, this time followed by a space, e.g.

```
123-First line of a multi line comment
Second line
234 Third line starts with some numbers
123 Last line of the multi line connection
```

Interpretation of the first digit

The first digit can take on one of five different values, and is used to specify the general acceptance of the requested action, as follows:

- *1xx – Positive Preliminary Reply*: The requested action is being initiated, expect another reply before proceeding with another command.
- *2xx – Positive Completion Reply*: The requested action has been successfully completed. A new command may be sent.
- *3xx – Positive Intermediate Reply*: The command has been accepted, but the requested action is being held, pending receipt of further information (e.g. a password is normally required once a username has been specified).
- *4xx – Transient Negative Completion Reply*: The command was not accepted, but the error condition is temporary, and the command may be requested again.
- *5xx – Permanent Negative Completion Reply*: The command was not accepted, nor should the command be sent again.

Interpretation of the second digit

The second digit is used to identify the general nature of the message, as follows:

- *x0x – Syntax*: Such replies refer to syntax errors, unimplemented or superfluous commands, etc.
- *x1x – Information*: Replies that fall under this category deal with requests for information, status or help.
- *x2x – Connections*: Replies referring to the control and data connections.
- *x3x – Authentication and accounting*: Replies relating to the login process and accounting procedures.
- *x4x – Unspecified*.
- *x5x – File system*: These replies indicate the status of the server file system, in relation to the requested transfer or other file system activities.

Interpretation of the third digit

The third digit provides a finer degree of graduation for each of the different function categories.

For example:

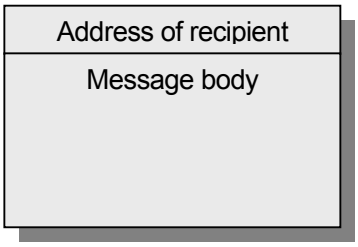
- 500 – Syntax error, command unrecognised.
- 501 – Syntax error in parameters
- 502 – Command not implemented
- 503 – Bad sequence of commands
- 504 – Command not implemented for that parameter.

Electronic Mail

In relation to the TCP/IP layered diagram, the process of creating, reading and managing emails is an application layer activity, whereas the process of sending an email from the sender to the recipient primarily occurs at the transport layer. Both processes will be considered.

History of Email

Email has existed for over two decades, being one of the first network applications. The first Email systems were nothing more than a FTP setup where the first line of the message (i.e. text stored in a file) contained the address of the recipient, followed by the message to be sent to the recipient.



Evidently this is a simplistic approach and suffered as there was no convenient means of sending a message to a group of people, nor did the sender of the message know if it ever arrived, etc.

In 1982, the ARPANET email proposals were outlined, and today still form the basis of Internet email. The proposals consisted of two main components: a transmission protocol (RFC 821) and a message format (RFC 822).

Breakdown of an email system

The functionality of an email system can be summarized as follows:

- **Composition:** corresponding to the process of message creation, including extra facilities such as an *address book*.
- **Transfer:** the process of moving a message from the sender to the recipient. Typically this is accomplished by sending the message to an intermediate machine, which then takes responsibility to forward it to the final destination.
- **Reporting:** informing the user if an email was correctly received, rejected, lost, etc.
- **Displaying:** presenting the content of an email to the user. For text-only emails this is mostly a trivial task, however, emails may also contain HTML, sound, pictures, etc.
- **Disposition:** extending to the management of emails, grouping into *mailboxes*, deleting messages, forwarding, *mailing lists*, etc.

Email systems typically consist of two different components: *user agents* and *message transfer agents*. User agents permit an email to be sent and read, whereas message transfer agents are responsible for moving messages towards their intended destination.

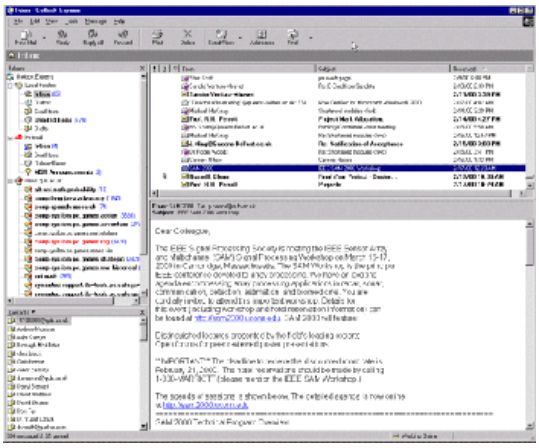
In what follows the user agent, message formats and message transfer protocols are further explored.

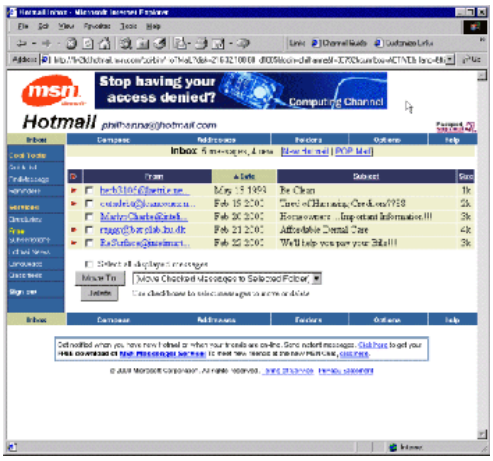
User agent

A user agent is normally a program (often called a *mail reader* or *mail client*) that accepts a variety of commands for composing, receiving, and replying to email messages. A wide variety of email readers exist; some text-only, others graphical (all, however, offer the same basic functionality).

The Outlook Express and Hotmail client interfaces are shown below:

Sending Email





In order to send an email the user typically provides a number of pieces of information, some optional, as follows:

Message Formats – RFC 822

The format of an email, as outlined by the RFC 822 standard is as follows:

Envelope
Header fields
Blank line
Body of message

← Defined in RFC 821, containing necessary info to allow the message to be sent over the network

The header consists of the following core information:

Header	Meaning
To:	Email address(es) of the primary recipient(s)
Cc:	Email address(es) of secondary recipient(s)
Bcc:	Email address(es) for blind carbon copies
From:	Creator of the message
Sender:	Email address of the sender of the message
Received:	Line added to by each transfer agent along the route to the destination
Return-Path:	Specifies a return-path back to the sender

In terms of deliver there is no difference between the To: and Cc: (Carbon copy) fields – any specified recipients receive the message. The difference is entirely psychology. The Bcc (Blind carbon copy) is used to send messages to a third party without the primary and secondary recipients knowing this.

The From: and Sender: fields might differ, for instance a secretary sends an email on behalf of their boss. These fields are needed in case the email cannot be delivered and must be returned to the sender.

The Received: and Return-Path: fields permit the route of an e-mail to be traced and, if necessary, a return route to the sender specified.

In addition to the above core information, an email's header may also contain a number of other optional fields, for example:

Header:	**Meaning:**
Date:	The data and time the message was sent
Reply-To:	Address to which replies should be sent
Message-Id:	Unique number referencing this message
In-Reply-To:	Message-Id to which this is a reply
References:	Other relevant Message-Ids
Keywords:	User chosen keywords
Subject:	Short summary of the email

The RFC 822 document specifies that users and email programs are permitted to invent new headers, provided the headers start with the string 'X-'. An example of some user defined labels are as follows:

```
X-Priority: 3
X-Msmail-Priority: Normal
X-Mailer: Microsoft Outlook Express 4.72
X-Mimeole: Produced By Microsoft MimeOLE V4.72
```

Multipurpose Internet Mail Extensions (MIME)

RFC 822 adequately caters for text only emails, however, it does not adequately support any email messages written in languages with accents (French, Spanish, etc.) or those that use a non-Latin alphabet (Russian, etc.), or even non-alphabetic languages (e.g. Chinese, etc.). Additionally, the message might consist of an audio or video clip.

In order to cater for these problems MIME was proposed (encapsulated in the RFC 1521 standard). MIME is based upon the RFC 822 standard, extending it to cater for non-ASCII messages. It is backwards compatible with RFC 822. MIME defines five new message headers, as follows:

MIME-Version:	Identifies the MIME version
Content-Description:	Human-readable string detailing the content of the message
Content-Id:	Unique identifier
Content-Transfer-Encoding:	How the body of the message is encoded
Content-Type:	Nature/type of the message.

The new headers are as follows:

- The *MIME-Version:* not only identifies what version of MIME is being used, but also that the message is in MIME format.
- The *Content-Id:* header is used to identify the message and takes the same format as the Message-Id: format (of RFC 822). The *Content-Description:* is intended for human consumption, identifying the contents of the message.
- A *Content-Transfer-Encoding:* field is needed as the message body might be encoded in a number of ways. The most commonly encountered are ASCII-7 bit encoding and ASCII-8 bit encoding (entailing that characters are either taken from the 7 or 8 bit ASCII character sets, and that lines are no longer than 1000 characters in length). Binary encoding uses 8 bit characters but does not impose an upper limit on the length of a line.

- The *Content-Type:* must be explicitly defined in the header (no default value is assumed), some of the basic content types are as follows:

Type	**Subtype**	**Description**
Text	Plain	Unformatted text
	Richtext	Text including simple formatting commands
Image	GIF	GIF format image
	JPEG	JPEG format image
Audio	Basic	Sound file
Video	MPEG	MPEG movie
Application	Octet-stream	Uninterpreted byte sequence
	Postscript	Printable document in Postscript
Message	RFC822	MIME RFC 822 message
	Partial	Message has been split for transmission
	External-body	Message itself must be fetched.

Aside: The above defines most of the different content types, but it is not exhaustive.

A description of each of the different data types now follows:

The '*Text*' category is for pure text messages (i.e. not containing anything other than text-based characters). The text/plain combination signifies that the message can be displayed as received, with no encoding or further processing necessary. The text/richtext subtype allows a simple markup language to be included (permitting **boldface**, *italics*, indentation, $_{\text{sub-}}$ $^{\text{super-}}$ scripting, etc.). It is the responsibility of the receiving system to interpret how the text should be displayed.

The '*Image*' grouping is used for sending pictures, officially supporting the GIF and JPEG standards.

The '*Audio*' and '*Video*' categories are for the transferal of sound and moving pictures. Note, that Video includes only the visual information, and not the soundtrack (which must be transmitted separately). MPEG is supported as the Video standard.

Note that the 'Application' type is a catch-all for formats that require external processing not covered by other MIME types. Octet-stream is nothing more than an unintrepreted stream of bytes, whereas, postscript refers to data encoded using Adobe's PostScript language. If necessary, it is possible to breakup an encapsulated message into pieces and send them separately.

Related to this is the External body subtype, where very long messages can be referred to via an FTP address, permitting the recipients browser to fetch it when needed. An example of an MIME message follows:

```
From: <outlkxpr@microsoft.com>
To: <New Outlook Express User>
Subject: Security Features in Outlook Express
Date: Tue, 2 Jun 1998 14:23:05 +0100
MIME-Version: 1.0
Content-Type: text/html; charset="iso-8859-1"
Content-Transfer-Encoding: quoted-printable

<html>
<head>
<title>Security Features in Outlook Express</title>
etc.
```

Message Transfer

Message transfer deals with the delivery of a message from the sender's computer to the recipient's mailbox. In principle this can be accomplished by establishing a transport connection and then transferring the message, a service which SMTP (Simple Mail Transfer Protocol) offers.

Simple Mail Transfer Protocol (SMTP)

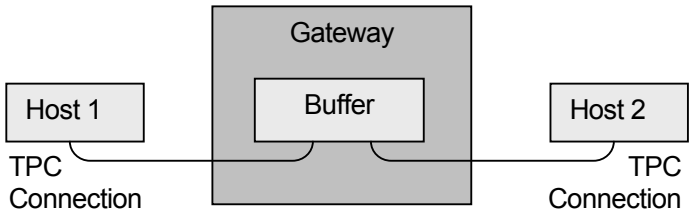
In order to send email over the Internet, the source machine establishes a TCP connection to port 25 on the destination machine. Listening to that port is an email daemon that understands SMTP (which is defined by RFC 821). The daemon attempts to receive the email and forward it to the appropriate mailbox. If a message cannot be delivered an error report is sent back to the sender.

The process is as follows:

Side	Action
Sender	Establishes a TCP connection on port 25 of the destination machine, and waits for the daemon to start 'talking'
Daemon	The daemon sends a line of text, identifying itself, and indicating if it is prepared to receive mail
Sender	If the daemon is not prepared to receive mail, or if there is no connection, then the connection is closed and an attempt made later. If the daemon will accept mail the sender specifies who the message is going to and who it is from.
Daemon	The daemon checks that it has a mailbox for the intended recipient, and if so, informs the sender to transmit the message.
Sender	Once permission to send the message has been obtained, the sender transmits the message
Daemon	Once the email has been received the daemon acknowledges receipt.
Sender	If any more email needs to be sent to this particular destination it is then transmitted.
Sender	Once all the email has been sent the connection is closed.
Daemon	Once the sender has indicated that no more mail is forthcoming, the connection is closed.

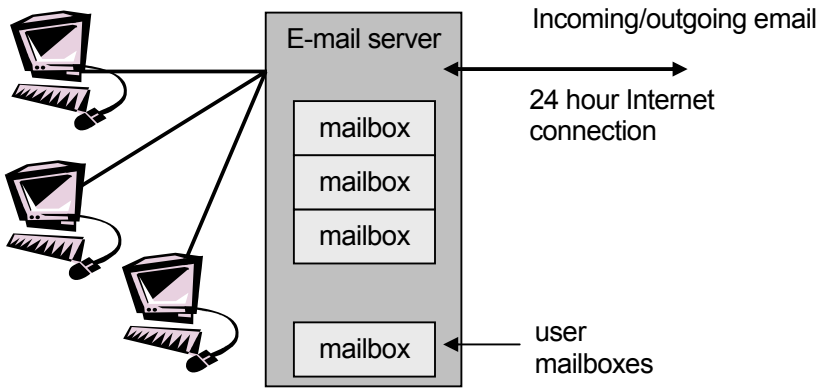
Email gateways

Often *email gateways* are used as an intermediate connection between the sender and the recipient (typically for those companies protected behind a firewall, where direct access cannot be made to a specific machine). Email gateways operate by accepting and storing incoming and outgoing mail before sending it towards the final destination.



Email servers

Having to establish a direct TCP connection between the source and destination computers may be problematic in the sense that emails can arrive 24 hours a day, entailing that a recipients computer must be always 'online'. As a solution email servers can be employed. The email server is solely dedicated to the transmission, receipt, storage, etc. of email.



A simple protocol for fetching email from a mailbox is POP3 (Post Office Protocol) which is defined in RFC 1225. POP3 has commands for logging in/out, fetching/deleting email.

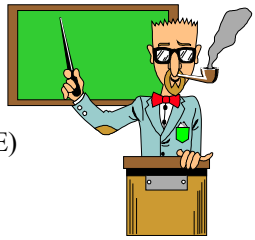
An alternative to POP3 is IMAP, which is increasingly employed within modern E-mail setups. IMAP stands for Internet Message Access Protocol and permits clients to access mail stored on an IMAP server as if it was a local server. For example, Email stored on an IMAP server can be manipulated from a desktop computer at home, a workstation in the office, or a notebook while traveling, all without the need to transfer files back and forth between the computers.

Practical 7

After this lecture you should explore the seventh practical pack which should enable you to investigate the material in this lecture.

Learning Outcomes

Once you have explored and reflected upon the material presented within this lecture and the practical pack, you should:



- Understand the role and functionality of FTP, including possessing knowledge of the client-server structure and communication protocol.
- Understand the structure of an E-mail as defined in RFC 822. In addition, understand how Multipurpose Internet Mail Extensions (MIME) enables a varied range of media types to be included within an E-mail.
- Understand the process by which E-mail transfer takes place.

More comprehensive details can be found in the CSC734 Learning Outcomes document.